UPI FRAUD DETECTION THROUGH MACHINE LEARNING AND FLASK FRAEWORK

A Project Report Submitted to

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

In Partial Fulfilment of the Requirements for the Award of the Degree of

MASTER OF COMPUTER APPLICATIONS

By

PRAVIN M R

Reg.No. RA2332241010304

Under the guidance of

Dr. S. ARUNARANI MCA., M.Phil., Ph.D.,

DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Kattankulathur – 603 203

Chennai, Tamilnadu

APRIL – 2025

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"UPI FRAUD DETECTION THROUGH MACHINE LEARNING AND FLASK FRAEWORK"** is a bonafide work carried out by **PRAVIN M R (RA2332241010304)** under my supervision for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by this students.

Dr. S. ARUNARANI

Assistant Professor

Department of Computer Applications

(GUIDE)

Dr. R. JAYASHREE

Associate Professor and Head

Department of Computer Applications

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION OF ASSOCIATION OF RESEARCH PROJECT WITH SUSTAINABLE DEVELOPMENT GOALS

This is to certify that the research project entitled **"UPI Fraud Detection Through Machine Learning and Flask Framework"** carried out by **Mr. Pravin M R** under the supervision of **Dr. S. Arunarani** in partial fulfillment of the requirement for the award of the Post-Graduation program has been significantly or potentially associated with **SDG Goal No. 16 (SIXTEEN) titled PEACE, JUSTICE, AND STRONG INSTITUTIONS.**

This study emphasizes the role of machine learning in detecting fraudulent transactions within UPI payment systems, ensuring secure digital transactions, and preventing financial crimes. By leveraging AI-driven fraud detection models and real-time analysis, the project contributes to enhancing cybersecurity, protecting users from financial threats, and fostering trust in digital financial services. The research aligns with global efforts to combat cyber fraud, strengthen financial regulations, and promote a safer and more transparent financial ecosystem.

SIGNATURE OF THE STUDENT **GUIDE**

HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

I are thankful to **Dr. T. R. Paarivendhar**, Chancellor, and **Prof. A. Vinay Kumar**, Pro Vice-Chancellor (SBL), SRM Institute of Science and Technology, who gave us the platform to establish me to reach greater heights.

I earnestly thank **Dr. A. Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science and Technology, who always encourage us to do novel things.

A great note of gratitude to **Dr. S. Albert Antony Raj**, Deputy Dean, Faculty of Science and Humanities for his valuable guidance and constant Support to do this Project.

I express our sincere thanks to **Dr. R. Jayashree**, Associate Professor and Head, for her support to execute all incline in learning.

It is our delight to acknowledge **Dr. S. Arunarani**, Assistant Professor, Department of Computer Applications, for her help, support, encouragement, suggestions, and guidance, which have helped to establish us to greater heights throughout the development phases of the project.

I convey our gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

Our gratitude to friends and people who are known and unknown to me who helped in carrying out this project work a successful one.

PRAVIN M R (RA2332241010304)

COMPANY LETTER

DLK CAREER DEVELOPMENT
IT SERVICES TECHNOLOGY CONSULTING

Date – 23 Jan 2025.

Project Confirmation Letter

This is to inform you that **Mr Pravin M.R (Register Number: RA2332241010304)** pursuing second year MCA COMPUTER APPLICATION at **SRM Institute of Science and Technology** is permitted to do her project work duration from **JAN 2025 to MAR 2025**. During the Project work the given details are confidential.

Woking on the project title - UPI Fraud Detection Through Machine Learning and Flask Framework

Yours Sincerely,

J Velmurugan
Project Team Head
DLK CAREER DEVELOPMENT

DLK CAREER DEVELOPMENT CHENNAI

PLAGIARISM CERTIFICATE

Dupli Checker

Date: 31-03-2025

Plagiarism Scan Report

0% Plagiarism	0% Exact Match
	0% Partial Match
100% Unique	

Words	228
Characters	1698
Sentences	8
Paragraphs	2
Read Time	2 minute(s)
Speak Time	2 minute(s)

Content Checked For Plagiarism

ABSTRACT
We place a strong emphasis on fortifying the security infrastructure of digital transactions within the Unified Payments Interface (UPI) framework by harnessing the power of machine learning algorithms, notably Random Forest and Logistic Regression, for fraud detection. Our methodology involves delving deep into transaction data, scrutinizing various parameters, and discerning anomalous patterns indicative of fraudulent behaviour. Through this meticulous analysis, our system endeavours to proactively mitigate the looming threat of fraudulent activities, thus reinforcing user confidence and trust in the UPI ecosystem. By employing a combination of advanced machine learning techniques and domain expertise, we aim to establish a robust defines mechanism against potential breaches, fostering a safer and more reliable environment for digital transactions. Through extensive experimentation, validation, and real-world simulation, we rigorously evaluate the efficacy of our proposed approach in identifying and thwarting fraudulent transactions. Our comprehensive testing framework encompasses diverse scenarios and real-time data streams, ensuring the scalability and adaptability of our fraud detection system. By showcasing the tangible benefits of our solution in terms of reduced financial losses and enhanced transaction security, we aspire to instil a sense of assurance among users and stakeholders, thereby bolstering the overall integrity and resilience of digital payment ecosystems. This project represents a pivotal step towards fortifying the foundation of digital transactions, paving the way for a future where secure and seamless financial transactions are the norm.

Matched Source

No plagiarism found

Check By: Dupli Checker

TABLE OF CONTENT

LIST OF FIGURES

LIST OF SCREENSHOTS

ABSTRACT

We place a strong emphasis on fortifying the security infrastructure of digital transactions within the Unified Payments Interface (UPI) framework by harnessing the power of machine learning algorithms, notably Random Forest and Logistic Regression, for fraud detection. Our methodology involves delving deep into transaction data, scrutinizing various parameters, and discerning anomalous patterns indicative of fraudulent behaviour. Through this meticulous analysis, our system endeavours to proactively mitigate the looming threat of fraudulent activities, thus reinforcing user confidence and trust in the UPI ecosystem. By employing a combination of advanced machine learning techniques and domain expertise, we aim to establish a robust defines mechanism against potential breaches, fostering a safer and more reliable environment for digital transactions. Through extensive experimentation, validation, and real-world simulation, we rigorously evaluate the efficacy of our proposed approach in identifying and thwarting fraudulent transactions. Our comprehensive testing framework encompasses diverse scenarios and real-time data streams, ensuring the scalability and adaptability of our fraud detection system. By showcasing the tangible benefits of our solution in terms of reduced financial losses and enhanced transaction security, we aspire to instil a sense of assurance among users and stakeholders, thereby bolstering the overall integrity and resilience of digital payment ecosystems. This project represents a pivotal step towards fortifying the foundation of digital transactions, paving the way for a future where secure and seamless financial transactions are the norm.

CHAPTER : 1

INTRODUCTION

1. Introduction

1.1 Problem Overview

This project represents a concerted effort to elevate the standard of digital payment security, with a particular focus on fortifying the integrity of transactions within the Unified Payments Interface (UPI) framework. Harnessing the capabilities of cutting-edge machine learning algorithms, including Random Forest and Logistic Regression, we are committed to deploying advanced techniques for the detection and prevention of fraudulent activities. Our primary objective is to cultivate a safer digital payment environment, one where users can transact with confidence and peace of mind, knowing that their financial assets are safeguarded against malicious actors.

Through a meticulous examination of transaction data, our approach is cantered on the identification of irregular patterns and anomalies that may signify potential instances of fraud. By scrutinizing various transaction parameters such as frequency, amount, location, and user behaviour, we endeavour to develop a comprehensive understanding of normal transactional patterns and discern deviations that may indicate fraudulent behaviour. Through this proactive analysis, our aim is not only to detect fraudulent transactions but also to pre-emptively intervene and prevent them from causing financial harm, thereby bolstering user trust and confidence in the UPI ecosystem.

The deployment of machine learning algorithms such as Random Forest and Logistic Regression enables us to leverage the power of predictive analytics to anticipate and counteract emerging threats in real-time. By continuously learning from historical transaction data and adapting to evolving patterns of fraud, our system is equipped to stay ahead of malicious actors and effectively mitigate the risks associated with digital transactions. Furthermore, by incorporating ensemble methods and anomaly detection techniques, we enhance

the robustness and accuracy of our fraud detection system, ensuring that even subtle deviations from normal behaviour are promptly identified and addressed.

Through rigorous testing, validation, and real-world deployment, we seek to demonstrate the tangible impact of our approach in fortifying the security posture of digital payments within the UPI framework. By showcasing the effectiveness of our fraud detection system in reducing instances of fraud and minimizing financial losses, we aim to instil a sense of confidence and trust among users, merchants, and financial institutions alike. Ultimately, this project endeavours to pave the way for a future where digital transactions are not only convenient and efficient but also inherently secure and resilient against emerging threats.

1.2 Artificial Intelligence

Artificial intelligence (AI) is the ability of a computer program or a machine to think and learn. It is also a field of study which tries to make computers "smart". As machines become increasingly capable, mental facilities once thought to require intelligence are removed from the definition. AI is an area of computer sciences that emphasizes the creation of intelligent machines that work and reacts like humans. Some of the activities computers with artificial intelligence are designed for include: Face recognition, Learning, Planning, Decision making etc.,

Artificial intelligence is the use of computer science programming to imitate human thought and action by analysing data and surroundings, solving or anticipating problems and learning or self-teaching to adapt to a variety of tasks.

1.3 Machine Learning

Machine learning is a growing technology which enables computers to learn automatically from past data. Machine learning uses various algorithms for building mathematical models and making predictions using historical data or

information. Currently, it is being used for various tasks such as image recognition, speech recognition, email filtering, Facebook auto-tagging, recommender system, and many more.

Machine Learning is said as a subset of artificial intelligence that is mainly concerned with the development of algorithms which allow a computer to learn from the data and past experiences on their own. The term machine learning was first introduced by Arthur Samuel in 1959. We can define it in a summarized way as: “Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed”.

A Machine Learning system learns from historical data, builds the prediction models, and whenever it receives new data, predicts the output for it. The accuracy of predicted output depends upon the amount of data, as the huge amount of data helps to build a better model which predicts the output more accurately.

Suppose we have a complex problem, where we need to perform some predictions, so instead of writing a code for it, we just need to feed the data to generic algorithms, and with the help of these algorithms, machine builds the logic as per the data and predict the output. Machine learning has changed our way of thinking about the problem.

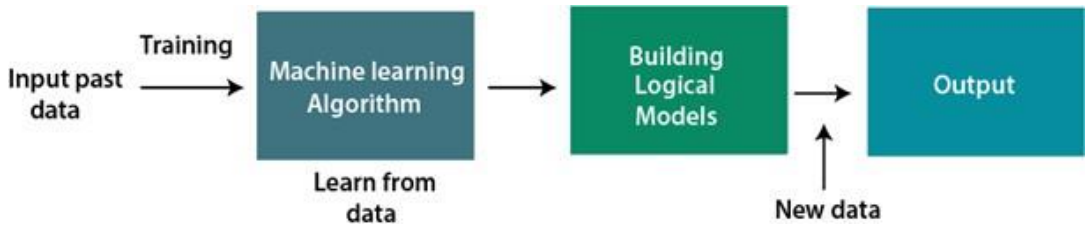


Fig 1. Block Diagram explains the working of Machine Learning Algorithm

1.3.1 Features of Machine Learning

Machine learning uses data to detect various patterns in a given dataset. It can learn from past data and improve automatically. It is a data-driven technology.

Machine learning is much similar to data mining as it also deals with the huge amount of the data.

Classification of Machine Learning. At a broad level, machine learning can be classified into three types

1) Supervised learning
2) Unsupervised learning
3) Reinforcement learning

1) Supervised Learning

Supervised learning is a type of machine learning method in which we provide sample labelled data to the machine learning system in order to train it, and on that basis, it predicts the output.

The system creates a model using labelled data to understand the datasets and learn about each data, once the training and processing are done then we test the model by providing a sample data to check whether it is predicting the exact output or not.

The goal of supervised learning is to map input data with the output data. The supervised learning is based on supervision, and it is the same as when a student learns things in the supervision of the teacher. The example of supervised learning is spam filtering. Supervised learning can be grouped further in two categories of algorithms

Classification **Regression**

2) Unsupervised Learning

Unsupervised learning is a learning method in which a machine learns without any supervision. The training is provided to the machine with the set of data that has not been labelled, classified, or categorized, and the algorithm needs to act on that data without any supervision. The goal of unsupervised learning is to restructure the input data into new features or a group of objects with similar patterns.

In unsupervised learning, we don't have a predetermined result. The machine tries to find useful insights from the huge amount of data. It can be further classifieds into two categories of algorithms.

- **Clustering**
- **Association**

1.4 ALGORITHMS

1.4.1 Random forest

Random forest is a Supervised Machine Learning Algorithm that is used widely in Classification and Regression problems. It builds decision trees on different samples and takes their majority vote for classification and average in case of regression.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing continuous variables as in the case of regression and categorical variables as in the case of classification. It performs better results for classification problems.

Working of Random Forest Algorithm

Before understanding the working of the random forest, we must look into the ensemble technique. Ensemble simply means combining multiple models.

Thus, a collection of models is used to make predictions rather than an individual model.

Ensemble uses two types of methods

1) **Bagging–** It creates a different training subset from sample training data with replacement & the final output is based on majority voting. For example, Random Forest.
2) **Boosting–** It combines weak learners into strong learners by creating sequential models such that the final model has the highest accuracy.

Steps involved in random forest algorithm

Step 1: In Random Forest n numbers of random records are taken from the data set having k number of records.

Step 2: Individual decision trees are constructed for each sample.

Step 3: Each decision tree will generate an output.

Step 4: Final output is considered based on Majority Voting or Averaging for Classification and regression respectively.

Important Features of Random Forest

- **Diversity-** Not all attributes/variables/features are considered while making an individual tree, each tree is different.
- **Immune to the curse of dimensionality-** Since each tree does not consider all the features, the feature space is reduced.
- **Parallelization-** Each tree is created independently out of different data and attributes. This means that we can make full use of the CPU to build random forests.
- **Train-Test split-** In a random forest we don't have to segregate the data for train and test as there will always be 30% of the data which is not seen by the decision tree.

- **Stability-** Stability arises because the result is based on majority voting/ averaging.

1.4.2 Logistic Regression

Logistic Regression is a fundamental algorithm in the realm of machine learning, primarily utilized for binary classification tasks, where the target variable can take only two possible outcomes. Despite its name, Logistic Regression is not a regression technique; instead, it's a classification algorithm that predicts the probability of an observation belonging to a particular class. This algorithm is widely employed due to its simplicity, interpretability, and efficiency, making it a cornerstone in various domains of data science and predictive modeling.

At its core, Logistic Regression models the relationship between one or more independent variables (often called features or predictors) and a binary dependent variable (the target or outcome). It achieves this by using the logistic function, also known as the sigmoid function, to transform the linear combination of the input features into a value between 0 and 1. This transformed value represents the probability of the input belonging to one of the two classes. The logistic function ensures that the output lies within the range of 0 to 1, making it suitable for modeling probabilities.

One of the key advantages of Logistic Regression lies in its interpretability. Unlike more complex machine learning algorithms, Logistic Regression provides clear insights into the influence of each feature on the predicted outcome. The coefficients associated with each feature indicate the direction and strength of their impact on the probability of belonging to a particular class. This interpretability makes Logistic Regression a valuable tool for understanding the underlying relationships between variables in a dataset.

Training a Logistic Regression model typically involves estimating the coefficients of the model that best fit the training data. This process is often achieved through optimization techniques such as gradient descent or maximum likelihood estimation. Once trained, the model can then make predictions on new data by calculating the probability of each observation belonging to the positive class (commonly labeled as 1). By comparing these probabilities to a predefined threshold (typically 0.5), the model classifies the observations into one of the two classes.

While Logistic Regression is a powerful algorithm, it has certain limitations. One major limitation is its assumption of linearity between the features and the log odds of the target variable. If the relationship is non-linear, Logistic Regression may not perform optimally, necessitating the use of more complex models. Additionally, Logistic Regression may struggle with datasets containing high levels of noise or multicollinearity among features. Despite these limitations, Logistic Regression remains a versatile and widely used algorithm in the field of machine learning, particularly in scenarios where interpretability and simplicity are paramount.

CHAPTER : 2

PROBLEM DEFINITION

2. Problem Definition

2.1 Defining the Business Problem

With the rapid expansion of digital payment systems, particularly through the Unified Payments Interface (UPI), fraudulent activities have become a pressing concern. Cybercriminals continuously exploit vulnerabilities in transaction processes, leading to financial losses and diminished user trust. Traditional fraud detection mechanisms, such as rule-based systems, often fail to keep pace with the evolving nature of fraud, necessitating the implementation of more advanced and intelligent solutions.

To combat this issue, leveraging machine learning algorithms offers a proactive approach to fraud detection. By employing models like Random Forest and Logistic Regression, transaction data can be analyzed in real time to identify suspicious patterns. These algorithms enable the detection of anomalies that may indicate fraudulent behavior, allowing for swift intervention before financial damage occurs. A robust fraud detection system ensures adaptability to new fraud tactics, improving overall transaction security.

Developing a scalable and efficient fraud detection mechanism enhances user confidence in digital transactions. By reducing fraudulent activities, financial institutions and payment platforms can maintain a secure ecosystem that fosters trust among users. The implementation of an advanced fraud prevention system within UPI not only mitigates risks but also strengthens the reliability of digital financial services, ultimately paving the way for a more secure and seamless transaction experience.

2.2 Business Requirements

To effectively combat fraudulent activities within the Unified Payments Interface (UPI) ecosystem, a robust fraud detection system must be implemented. This system should leverage machine learning algorithms such as Random Forest

and Logistic Regression to analyze transaction data in real time. By identifying anomalous patterns indicative of fraud, the system will enhance security and mitigate financial risks. The ability to detect fraud proactively ensures that both users and financial institutions can operate with greater confidence in digital transactions.

A critical requirement for this system is comprehensive data collection and processing. Transaction data, including user details, transaction amounts, timestamps, and frequency, must be gathered and preprocessed to improve accuracy. Proper data cleaning and feature selection are essential for training machine learning models effectively. The fraud detection models must be optimized to minimize false positives and false negatives, ensuring that legitimate transactions are not mistakenly flagged while fraudulent ones are accurately identified.

Scalability and adaptability are also key considerations in designing an efficient fraud detection system. The system must be capable of handling large transaction volumes without performance degradation. Additionally, it should continuously learn from new data to adapt to evolving fraud tactics. Implementing real-time alert mechanisms will further enhance security by notifying users and financial institutions of suspicious transactions and assigning risk scores to classify threats based on severity.

Beyond technical implementation, ensuring compliance with security regulations and fostering user trust are vital. The system must adhere to industry standards for data protection, incorporating encryption and privacy measures to safeguard sensitive information. By minimizing disruptions to legitimate transactions and demonstrating tangible benefits such as reduced fraud cases and financial losses, the system will reinforce confidence in UPI payments. A well-designed fraud detection mechanism will ultimately strengthen the integrity and resilience of the digital payments ecosystem.

2.3 Literature Survey

[1] Title: Leveraging Machine Learning Algorithms for Fraud Detection and Prevention in Digital Payments: A Cross Country Comparison

Author: Ruchika Gupta, Priyank Srivastava, Harish Kumar Taluja, Sanjeev Sharma, Shyamal Samant, Sanatan Ratna & Aparna Sharma

Description: The COVID-19 outbreak has brought unprecedented shifts for the global financial system and has hastened the adoption of digital payments. More insidious fraud schemes have risen as a result of these shifts, creating fertile ground for all sorts of financial fraud. Therefore, this paper presents a complete review of the fraud environment in digital payments, as well as the various approaches used by regulatory agencies in various nations throughout the world. This paper further describe how machine learning algorithms may be used to detect and prevent digital payment fraud in the post-pandemic period. Finally, some of the major obstacles and promising possibilities are discussed in order to give future inspiration for intelligent payment fraud detection.

[2] Title: ANALYSIS OF ONLINE INTRUSION DETECTION MODELS TO INCORPORATE SECURED DIGITAL CASH TRANSACTION IN MOBILE SMART SYSTEMS

Author: R. Bhuvaneswari, V. Vasanthi, M. Paul Arokiadass Jerald I. Benjamin Franklin

Description: The major Objective of this research paper is to design the Mobile Smart Device Digi Cash Intrusion Detection Framework (MSDDID) for assessing Intrusion Detection (ID) techniques and evaluating ID parameters that has to be rectified for enhancing the security of Digital Cash Transactions in Mobile Smart devices. The Research examined the Intrusion Detection dataset with 41 predictive features and 1 class feature for evaluating prediction in its novel form. The Framework was examined in WEKA with RapidMiner for

analysis. The Results of classifiers Decision Table (98.7%), Random Forest Tree (99.79%), AdaBoost (94.37%), CART Model (99.61%), LazyIBK (99.44%), Naïve Bayesian (89.66%) signified that Smart devices security in Digi cash transactions could be predicted with refinement of data during transaction as deployed in this research work. The cluster analysis again conformed that num_root, su_attempted and num_compromised were the three parameters predominantly used for intrusions in the network and has to be addressed in the model.

[3] Title: Online Transactions Fraud Detection using Machine Learning

Author: Ms. Kishori Dhanaji Kadam, Ms. Mrunal Rajesh Omanna, Ms. Sakshi Sunil Neje, Ms. Shraddha Suresh Nandai.

Description: Nowadays Digital transactions are rapidly increasing as it results in increasing online payment frauds too. In fact, according to the Reserve Bank of India, comparing March 2022 to March 2019, digital payments have risen in volume and value by 216% and 10%, respectively. People are starting to go all-in with digital transactions, but one can't deny the security issues that loom, and know-how when it comes to online payments. Few years ago, we could have barely seen the online payment, but today UPI payment QR code installed at doorstep. This invited the hoaxers and attackers to develop fraudulent transactions and fool people for some amount of money. Fortunately, the online transactions are monitored and hence could be analysed using the latest tools. In this system, an attempt is made to develop a machine learning model to identify fraudulent transactions in a transaction's dataset.

[4] Title: AI-POWERED SECURITY IN INDIA'S UPI TRANSACTIONS: EVALUATING TRANSACTION VOLUMES, FRAUD INCIDENTS, AND MITIGATION STRATEGIES

Author: Mr. A. S. Durwin, Dr. T. S. R. Vijay Janani, Dr. K. Deepa, Mr. G. Sai Bharath, Ms. R. Srinidhi

Description: The method of payments has evolved so much that once they were so manual and had to be made through cashiers or tellerss. The modern payment methods are so easy and efficient that they could be done with just a tap of a phone. In this study we would be looking at two such innovative methods of payment and evaluate them on the basis of their adaptability in preferentially diverse markets. The study has made a humble attempt to incorporate the role played by AI in such payment methods and the impact caused by the role on the users. NFC, which is short for near-field communication, is a technology that allows devices like phones and smart watches to exchange small bits of data with other devices and read NFC-equipped cards over relatively short distances. The technology behind NFC is very similar to Radio-Frequency Identification (RFID) commonly used in the security cards and keychain fobs, popularly experienced at offices, hotels, gyms, etc,. Unified Payments Interface (UPI) is a system that powers multiple bank accounts into a single mobile application (of any participating bank), merging several banking features, seamless fund routing & merchant payments into one hood. It also caters to the “Peer to Peer” collect request which can be scheduled and paid as per requirement and convenience. Each Bank provides its own UPI App for Android, Windows and iOS mobile platform(s). The paper is based partially on secondary data and partially on primary data to assess the factors influencing the adoption of the payment systems by Indian users, to analyse the impact of AI technology and banking services and finally suggest certain policy measures to provide secured and inclusive digital transformation in banking transactions.

[5] Title: An Innovation Detection of Vulnerabilities for Digital Transactions in Financial Institutions Using Cyber Security Framework

Author: R. Sangeetha, R. Priscilla Joy, M. Denisha, Julia Punitha Malar Dhas

Description: Cyber security is an important and growing challenge in the world today. As technology advances, so do the potential threats posed by malicious actors. Cyber security is the practice of protecting networks, systems, and programs from digital attacks. These attacks are usually aimed at accessing, changing, or destroying sensitive information, extorting money from users, or interrupting normal business processes. Cyber security threats come from a variety of sources, including activists, criminals, fo reign governments, and terrorists. All of these actors are constantly developing new tools and techniques for attacking systems. As a result, organizations must stay on top of the latest developments in order to protect themselves from these threats. One of the biggest challenges in cyber security is defending against the ever-evolving threats posed by malicious actors. To do this, organizations must have a comprehensive security strategy in place. In this paper, a cyber-security framework has proposed to identify the cyber threats. This strategy should include preventative measures such as firewalls, monitoring systems, and patch management. It should also include detection and response measures, such as incident response plans and digital forensics. Another challenge is staying ahead of the attackers. Attackers are constantly using new techniques and tools, making it difficult to defend against them. Organizations must stay up- to-date with the latest.

CHAPTER : 3

SYSTEM CONFIGURATION

3. System Configuration

3.1 Hardware Specifications

Processor : Intel Core i5/i7 or equivalent (minimum 2.5 GHz, quad-core)

RAM : 8 GB (minimum), 16 GB or more recommended for better performance when handling large datasets and model training.

Hard Disk : 256 GB SSD (512 GB or more recommended for faster storage access and space for dataset and project files).

GPU : Optional but recommended if you're running more complex models (e.g., ANN) locally. If not, Google Colab provides GPU resources.

Display : 1080p or higher resolution for effective data visualization and project management.

Network : Stable high-speed internet connection for running code on cloud-based platforms like Google Colab, and for using cloud storage services.

3.2 Software Specifications

Operating System : Windows 10/11, macOS, or Linux (depending on your local machine; Google Colab abstracts much of this away for cloud usage).

Development Environments

Google Colab (Gemini) : Primary environment for machine learning model training and experimentation. Colab offers pre-installed libraries and access to GPU resources, which is ideal for machine learning tasks like flight delay prediction.

Visual Studio Code (VSS) : Used for writing and managing code, particularly for the Flask framework and HTML frontend.

Web Development

Flask : For building the web-based application that hosts the machine learning model for flight delay prediction.

HTML : For creating the user interface where users can input flight details and receive delay predictions.

CSS/JavaScript : Optional for additional UI/UX enhancements.

3.3 Software Description

Google Colab : Colab offers a cloud-based Jupyter notebook environment with the advantage of free access to GPUs and TPUs. In the context of flight delay prediction, it will be used for exploratory data analysis (EDA), model training (using machine learning libraries like Scikit-learn, TensorFlow, or Keras), and model evaluation. Colab's integration with Google Drive allows for easy data storage and retrieval.

Visual Studio Code (VSS) : A powerful and flexible IDE used to develop the application's backend (Flask) and frontend (HTML). With features like debugging, Git integration, and extensions for Python, Flask, and HTML, VSS is suitable for managing the entire project.Flask: This web framework will be used to develop the backend of the application, allowing users to interact with the trained machine learning model. Flask will handle user inputs (flight details) and return predictions from the model, integrating the model built in Colab.

HTML : Used for designing the front end of the web application. The interface will collect inputs like flight date, departure time, and weather conditions from users and display the model's delay predictions.

CHAPTER : 4

SYSTEM ANALYSIS

4. System Analysis

4.1 Existing System

The current UPI fraud detection system predominantly relies on manual methodologies and rule-based strategies, which have several inherent limitations. These systems depend on predefined rules and human intervention to identify fraudulent activities, making the process time-consuming and inefficient. Financial institutions must allocate significant resources to manually investigate flagged transactions, leading to delays in detecting and mitigating fraud. This reliance on manual analysis introduces potential inconsistencies and errors, weakening the overall effectiveness of fraud detection.

One of the critical shortcomings of rule-based fraud detection is its inability to adapt to new and sophisticated fraud tactics. Cybercriminals continuously evolve their methods, finding ways to bypass static rules designed to detect fraudulent transactions. Since rule-based systems operate on predefined patterns, they often fail to recognize novel fraud schemes, allowing malicious actors to exploit system weaknesses. This rigidity makes traditional fraud detection methods inadequate in a rapidly changing digital payment landscape.

Scalability is another major issue with the existing system. As digital transactions continue to grow exponentially, manually reviewing large datasets becomes increasingly unmanageable. Traditional fraud detection mechanisms struggle to analyze vast amounts of transaction data efficiently, leading to delayed responses to fraudulent activities. The inability to process real-time transaction data effectively compromises the security of the UPI ecosystem, leaving users vulnerable to financial losses.

Furthermore, the dependency on manual methods results in high operational costs for financial institutions. The need for extensive human intervention and dedicated fraud investigation teams adds to the overall expenses,

making fraud detection an unsustainable practice in the long run. Given these limitations, there is a pressing need for a more efficient, automated, and scalable fraud detection system that can adapt to emerging fraud patterns while minimizing human intervention.

4.2 Proposed System

To address the limitations of manual and rule-based fraud detection, a machine learning-driven approach is proposed. This system leverages advanced algorithms such as Random Forest and Logistic Regression to analyze transaction data in real time, identifying fraudulent activities based on nuanced patterns rather than predefined rules. By automating fraud detection, the proposed system ensures faster, more accurate identification of suspicious transactions, reducing reliance on human intervention and improving overall security.

Machine learning models have the ability to continuously learn from historical and real-time transaction data, allowing them to detect emerging fraud tactics that traditional systems might overlook. Unlike static rule-based methods, machine learning algorithms adapt dynamically to new fraud patterns, making them more effective in identifying sophisticated fraudulent schemes. This adaptability ensures that the fraud detection system remains robust against evolving threats in the digital payments landscape.

Another key advantage of the proposed system is its scalability. Machine learning models can process vast amounts of transaction data quickly and efficiently, enabling real-time fraud detection even as transaction volumes increase. By integrating this automated system into the UPI framework, financial institutions can proactively detect fraudulent activities, preventing financial losses and enhancing user trust in digital transactions.

Additionally, the implementation of a machine learning-driven fraud detection system reduces operational costs associated with manual investigations.

Automating the fraud detection process allows financial institutions to allocate resources more effectively, improving overall efficiency. By enhancing accuracy, speed, and scalability, the proposed system significantly strengthens UPI security, ensuring a safer digital payment ecosystem for users and stakeholders.

4.3 Feasibility Study

1) Technical Feasibility

The proposed machine learning-based fraud detection system is technically feasible as it leverages well-established algorithms such as Random Forest and Logistic Regression. These models can be effectively trained using historical transaction data to identify fraudulent patterns. Cloud computing and big data technologies provide the necessary computational power to process vast amounts of real-time transaction data efficiently. Additionally, integration with existing UPI infrastructure is achievable, as many financial institutions already utilize AI-driven solutions in various domains.

2) Economic Feasibility

From a financial standpoint, the proposed system presents a cost-effective solution compared to the high operational costs of manual fraud detection. While initial investments in data infrastructure, model development, and deployment may be required, the long-term benefits outweigh these costs. Automated fraud detection reduces expenses related to human intervention, improves fraud prevention efficiency, and minimizes financial losses caused by fraudulent transactions. The return on investment (ROI) is significant, as enhanced security fosters user confidence and encourages greater adoption of digital payments.

3) Operational Feasibility

The transition to an automated fraud detection system aligns with the operational needs of financial institutions and regulatory bodies. Employees can

be trained to monitor machine learning models, interpret alerts, and refine fraud detection strategies without being burdened by manual transaction reviews. The system's ability to generate real-time fraud alerts allows for quick responses, enhancing overall operational efficiency. Furthermore, users benefit from improved transaction security, reinforcing trust in UPI payments.

4) Legal and Security Feasibility

Compliance with financial regulations and data protection laws is essential for implementing a machine learning-driven fraud detection system. The proposed solution ensures data privacy by incorporating encryption and secure authentication mechanisms. Regulatory approval can be obtained by demonstrating the system's ability to reduce fraud while maintaining user confidentiality. By adhering to legal and security standards, the fraud detection system strengthens digital payment security while ensuring compliance with industry regulations.

CHAPTER : 5

SYSTEM DESIGN

5. System Design

5.1 System Architecture

The proposed UPI fraud detection system is designed to efficiently analyze real-time transactions using machine learning techniques, ensuring enhanced security and reduced financial losses. The architecture consists of multiple interconnected components that work together to detect fraudulent transactions dynamically. The process begins with **User Transactions**, where transaction data—including user details, transaction amount, timestamps, and geolocation—is collected in real-time from the UPI platform. This data is then passed through the **Data Preprocessing Module**, where it undergoes cleaning, normalization, and transformation to remove inconsistencies and ensure high-quality input for analysis. The **Feature Extraction & Engineering** component identifies key fraud indicators, such as transaction frequency, abnormal amounts, and suspicious geolocations, to improve detection accuracy.

The core of the system lies in the **Machine Learning Model**, which utilizes algorithms such as **Random Forest and Logistic Regression** to analyze transaction patterns. The model assigns a **risk score** to each transaction based on historical fraud trends and real-time behavioral analysis, classifying transactions as either legitimate or fraudulent. The **Real-Time Fraud Detection Engine** continuously monitors transactions and applies these models to identify suspicious activity. When a fraudulent transaction is detected, the **Alert & Response Mechanism** is triggered, sending real-time notifications to users and financial institutions. Immediate actions such as transaction blocking, additional authentication requests, or account suspension can be implemented to prevent financial losses.

To ensure adaptability, the **Model Training & Continuous Learning** module continuously updates fraud detection models using newly observed fraud

patterns, making the system dynamic and resilient to evolving threats. The **Reporting & Analytics Dashboard** provides stakeholders with insights into fraud trends, model performance, and system effectiveness, allowing for informed decision-making. This architecture ensures a **highly scalable, automated, and adaptive fraud detection system** that minimizes manual intervention, enhances detection accuracy, and fosters user trust in digital transactions.

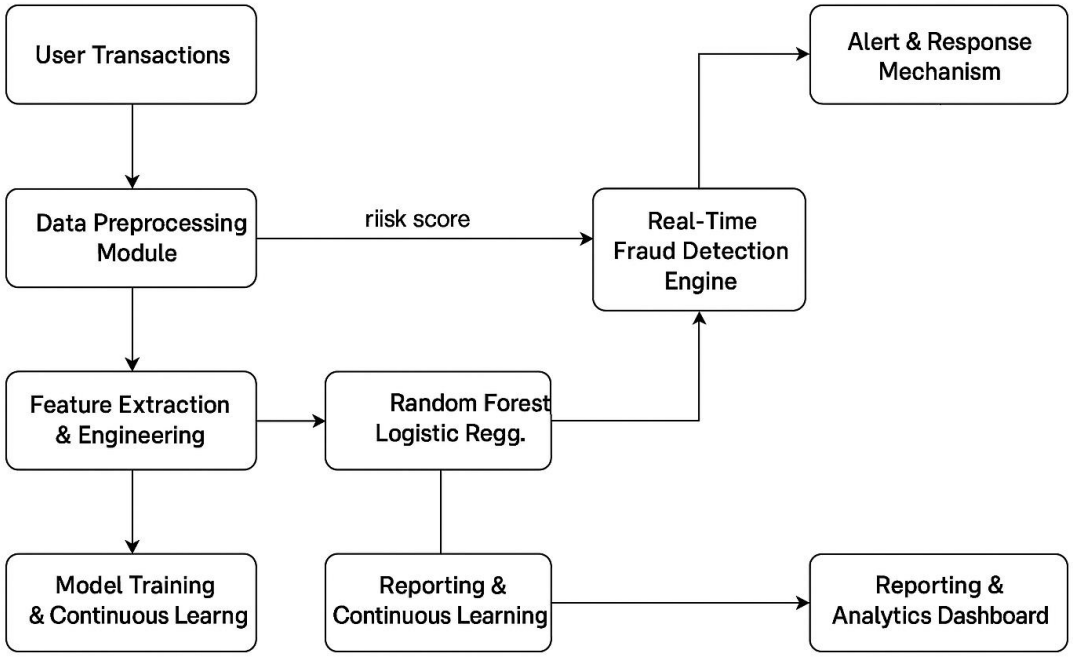


Fig 2. Architecture of Proposed Model

5.2 Use Case and Activity Diagrams

Use Case Diagram

The Use Case Diagram represents the interaction between the user, financial institutions, and the fraud detection system. It highlights various functionalities, such as transaction processing, fraud detection, alert generation, and continuous model learning.

Actors

User – Initiates transactions and receives alerts for suspicious activities.

Fraud Detection System – Processes transactions, applies machine learning models, and detects fraudulent activities.

Financial Institution – Monitors flagged transactions, takes necessary actions, and updates the fraud detection system.

Use Cases

Initiate Transaction – The user initiates a payment through UPI.

Process Transaction – The system validates transaction data and checks for anomalies.

Fraud Detection – The machine learning model analyzes transaction risk and classifies it.

Generate Alert – If a transaction is suspicious, an alert is sent to the user and financial institution.

Take Action – The financial institution investigates and either approves or blocks the transaction.

Model Update – The fraud detection system updates its model using new fraud patterns.

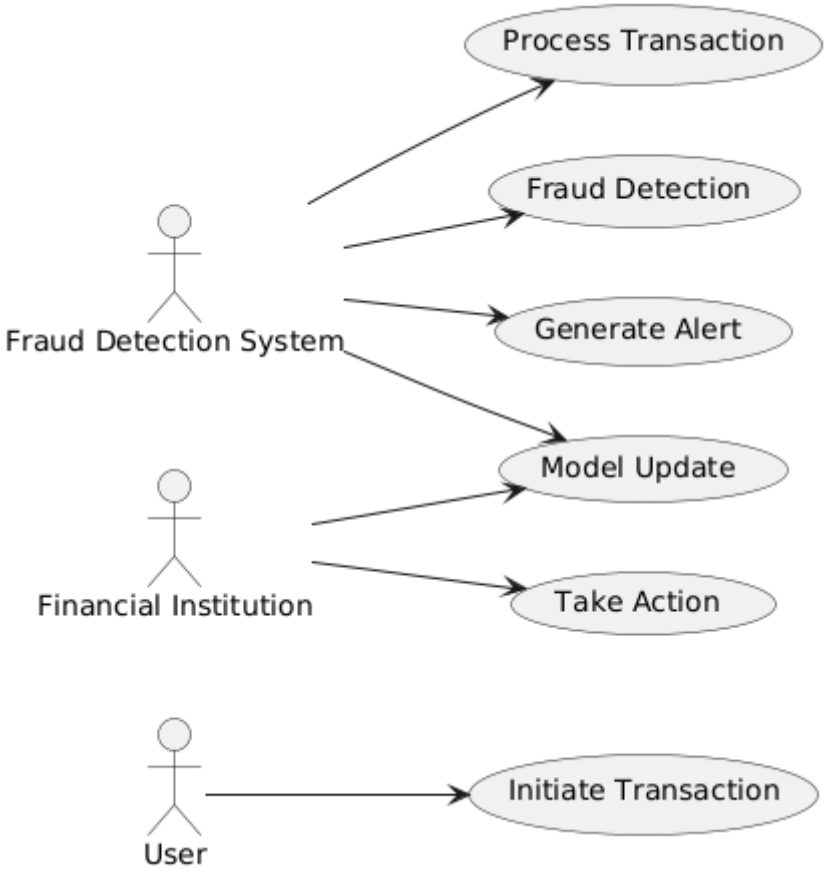


Fig 3. Use Case Diagram

Activity Diagrams

An Activity Diagram shows the flow of activities or tasks within a system or process. It represents the sequence of actions or workflows, including decision points and parallel processes. Activity diagrams are useful for visualizing the logic of a process, showing how different tasks or steps are connected.

This activity diagram simplifies the machine learning workflow while keeping the main steps involved in the process. Here's a breakdown of the key components and actions,

The Activity Diagram provides a visual representation of the fraud detection process, detailing the flow of activities from transaction initiation to

fraud detection and mitigation. The system ensures a seamless and secure UPI transaction experience by incorporating machine learning algorithms for real-time analysis and decision-making.

User Initiates a Transaction

- The process begins when a user initiates a payment through a UPI platform.
- The transaction request includes details such as sender and receiver information, transaction amount, timestamp, geolocation, and device ID.
- This data is immediately sent to the fraud detection system for verification.

Transaction Data Collection

- The fraud detection system receives the transaction request and logs the data for further analysis.
- The system extracts key attributes, including transaction amount, frequency of transactions, location, and past transaction history of the user.
- This data serves as the input for fraud analysis and detection.

Data Preprocessing and Feature Extraction

- The raw transaction data undergoes preprocessing to remove inconsistencies, missing values, and duplicate entries.
- Feature extraction is performed, where crucial fraud indicators such as unusual transaction amounts, rapid transaction frequency, and device changes are identified.
- The data is normalized to ensure consistency, allowing the machine learning model to make accurate predictions.

Machine Learning Model Analysis

- The processed data is fed into the trained fraud detection model, which applies algorithms such as **Random Forest and Logistic Regression** to classify the transaction.
- The model evaluates transaction behavior, compares it with historical patterns, and assigns a **fraud risk score** based on predefined thresholds.
- The system determines whether the transaction is **legitimate or fraudulent**.

Fraud Risk Score Evaluation

- If the fraud risk score is **below the threshold**, the system considers the transaction safe and approves it automatically.
- If the fraud risk score is **above the threshold**, the transaction is flagged as potentially fraudulent, and an alert is triggered.

Alert Generation for Suspicious Transactions

- If a transaction is flagged, the system generates an **alert notification** to both the user and the financial institution.
- The notification contains transaction details, risk score, and the reason for flagging (e.g., unusual location, high transaction amount, or device mismatch).

Financial Institution Review and Decision-Making

- The financial institution manually reviews flagged transactions to determine their legitimacy.
- Based on investigation results, they can choose to **approve, reject, or request additional verification** from the user.
- In some cases, the user may be asked to provide biometric authentication or an OTP (One-Time Password) for verification.

Final Decision: Approve or Block Transaction

- If the institution finds the transaction **legitimate**, it is approved, and the payment is processed successfully.
- If the transaction is deemed **fraudulent**, it is blocked to prevent financial loss.
- The user receives a confirmation of either approval or rejection.

Model Update and Continuous Learning

- The fraud detection system **updates its machine learning models** based on the latest fraud cases and transaction data.
- New fraud patterns are incorporated into the model to enhance detection accuracy.
- The system continuously improves to adapt to evolving fraud tactics, ensuring **real-time security and adaptability**.

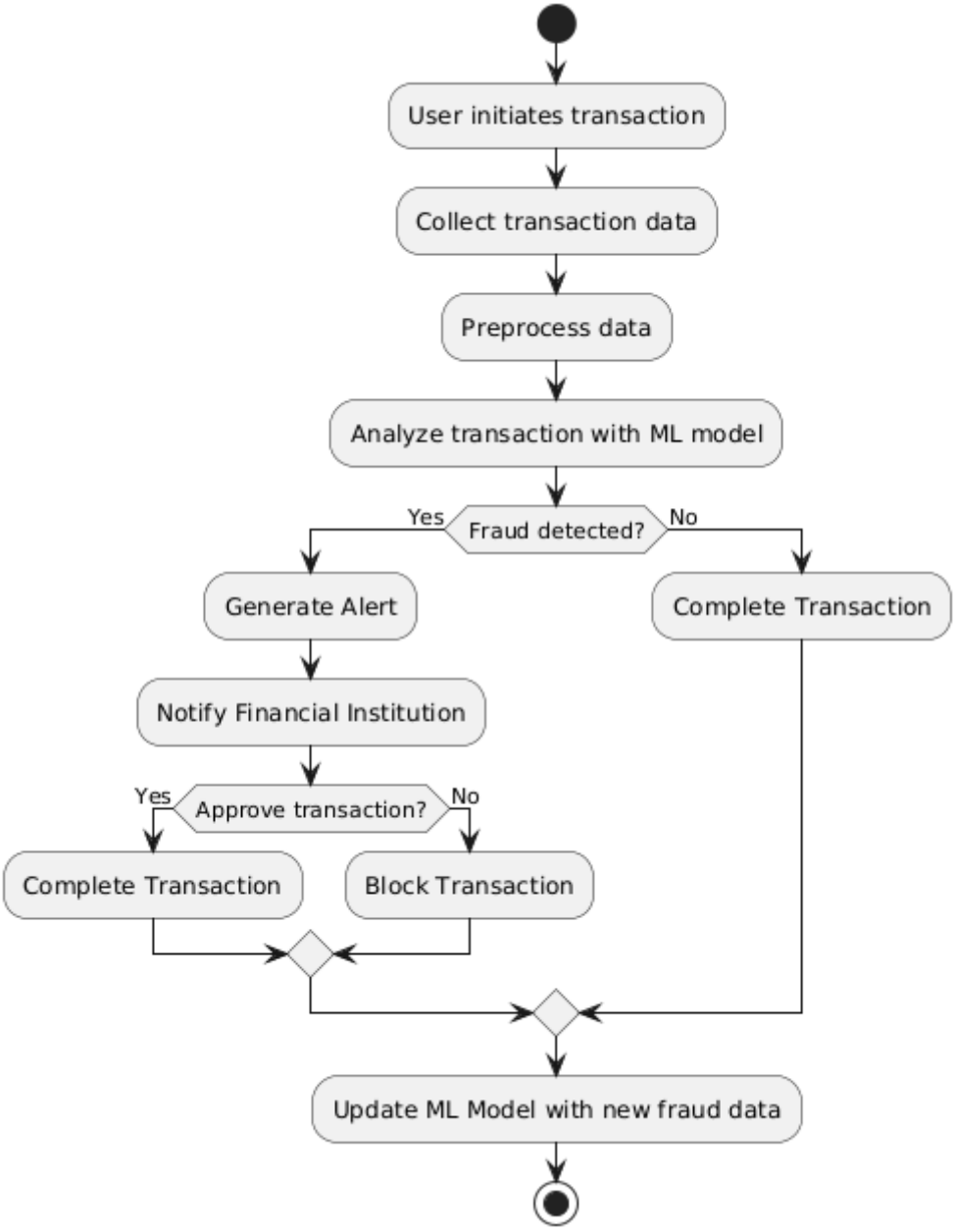


Fig 4. Activity Diagram

CHAPTER : 6

PROJECT FLOW

6. Project Flow

6.1 Data Collection

- The dataset used for this project is in CSV format.
- The data was downloaded from Kaggle, containing transactional records including timestamps, transaction amounts, sender-receiver details, and fraud indicators.

6.2 Data Preprocessing

- Handling null values by either removing them or using imputation techniques.
- Removing duplicate entries to avoid biased predictions.
- Handling outliers using statistical methods to improve model accuracy.
- Encoding categorical variables to convert non-numeric data into a machine-readable format.

6.3 Exploratory Data Analysis (EDA)

- Displot – Used to analyze the distribution of numerical features.
- Scatter Plot – Helps visualize relationships between different transaction attributes.
- Boxplot – Detects outliers in transaction amounts and frequency.
- Heatmap – Identifies feature correlations using a color-coded matrix.

6.4 Splitting of Data

- Training Set (80%) – Used to train the machine learning models.
- Testing Set (20%) – Used to evaluate model performance on unseen data.

6.5 Model Deployment

- Random Forest – A robust ensemble learning method for classification.
- AdaBoost – Boosting algorithm that improves weak classifiers.

- CatBoost – Optimized for categorical data and faster training.
- XGBoost – Highly efficient gradient boosting model.
- LightGBM – A fast and scalable boosting algorithm for large datasets.

6.6 Model Evaluation, Model Deployment & User Interaction (GUI)

- Accuracy Score – Measures the proportion of correctly classified transactions.
- Confusion Matrix – Evaluates the number of true positives, false positives, true negatives, and false negatives.
- Precision, Recall, and F1-Score – Helps determine model efficiency in fraud detection.
- The best-performing model is selected based on evaluation metrics.
- The model is saved and integrated into a Flask-based web framework.
- Run the app.py file to start the web application.
- Copy the generated local server link and open it in Chrome.
- Enter transaction details in the UI and click "Predict".
- The system analyzes the input and displays the fraud prediction on the screen.

6.1 Data Collection

- The dataset used for this project is in CSV format.
- The data was downloaded from Kaggle, containing transactional records including timestamps, transaction amounts, sender-receiver details, and fraud indicators.

The dataset used for this project is in CSV format.

Data collection is the foundation of any machine learning project, particularly in fraud detection systems where the quality and structure of data significantly impact the accuracy of predictions. In this project, the dataset is stored in **Comma-Separated Values (CSV) format**, which is widely used for structured data representation due to its simplicity and compatibility with various data analysis tools and programming environments.

Using a **CSV file format** offers multiple advantages. Firstly, it allows easy data manipulation and processing in tools like **Python (pandas), Excel, SQL, and Google Sheets**. Since CSV files are **lightweight and human-readable**, they can be opened and edited using a simple text editor if necessary. Furthermore, most machine learning libraries, such as **Scikit-learn, TensorFlow, and PyTorch**, support reading CSV files directly, making it seamless to load data into analytical models.

Another key reason for using CSV is **its structured tabular format**, which ensures that transactional records are stored in rows and columns. Each row in the dataset represents an individual transaction, while columns represent different attributes such as transaction ID, sender ID, receiver ID, transaction amount, timestamp, transaction status, and whether the transaction was fraudulent or legitimate. This structure allows for easy indexing, searching, filtering, and transformation of data, making it ideal for fraud detection analysis.

CSV files also allow for **easy integration with databases and cloud storage systems**, which is essential when dealing with large datasets. Since fraud detection systems require real-time analysis, storing data in a structured format like CSV helps **quickly retrieve relevant transactions for processing**. Additionally, it facilitates smooth **data preprocessing, visualization, and modeling** by ensuring compatibility with various Python libraries like pandas, NumPy, and Matplotlib.

The Data Was Downloaded from Kaggle, Containing Transactional Records Including Timestamps, Transaction Amounts, Sender-Receiver Details, and Fraud Indicators

The dataset used in this project was sourced from **Kaggle**, a widely recognized online platform that provides datasets for data science and machine learning projects. Kaggle hosts a variety of datasets contributed by organizations and researchers, allowing developers to work with **real-world transactional data**.

Downloading the dataset from Kaggle ensures that the data is **authentic, diverse, and representative of real-world financial transactions**. The dataset contains **various transactional records**, including timestamps, transaction amounts, sender-receiver details, and fraud indicators. These attributes are crucial in detecting fraudulent patterns and training machine learning models effectively.

The **timestamp column** records the exact time and date of each transaction. This helps in identifying suspicious transaction behaviors, such as multiple transactions occurring within an unusually short period or transactions happening at odd hours (e.g., late at night). Fraudsters often exploit off-peak hours to carry out illicit activities, making timestamps an essential feature in fraud detection.

The **transaction amount column** records the value of each transaction. Unusually high-value transactions or frequent small transactions (structuring or smurfing) may indicate fraudulent activity. Analyzing transaction amounts helps identify suspicious money transfers that deviate from a user's typical spending pattern.

The **sender and receiver details** provide information on the parties involved in each transaction. Fraudulent transactions often involve unknown or high-risk accounts that have no prior transaction history with the sender. Additionally, detecting **multiple transactions to different receivers from the same sender in a short period** can signal money laundering or scam activities. By analyzing sender-receiver relationships, the system can flag **potential fraud rings** and prevent financial losses.

The **fraud indicator column** specifies whether a transaction is fraudulent or legitimate. This is a crucial feature for supervised learning algorithms, as it helps train the model to distinguish between **normal and fraudulent transactions**. The fraud indicator allows machine learning models such as **Random Forest, AdaBoost, CatBoost, XGBoost, and LightGBM** to learn from historical fraud patterns and improve their accuracy in predicting fraudulent activities.

In addition to these core features, the dataset may also include other attributes like **transaction mode (e.g., UPI, bank transfer, credit card), device ID, geolocation, and user behavior metrics**. These additional factors enhance the fraud detection system by providing more contextual insights into each transaction.

By using a **real-world dataset from Kaggle**, this project ensures that the fraud detection system is trained on **genuine financial transactions**, making it more effective and reliable. The dataset serves as a **comprehensive resource for**

developing, training, testing, and optimizing machine learning models to enhance UPI security and protect users from fraudulent activities.

6.2 Data Preprocessing

- Handling null values by either removing them or using imputation techniques.
- Removing duplicate entries to avoid biased predictions.
- Handling outliers using statistical methods to improve model accuracy.
- Encoding categorical variables to convert non-numeric data into a machine-readable format.

Data preprocessing is a crucial step in preparing raw transaction data for fraud detection. Since UPI transaction records contain various inconsistencies, missing values, duplicate entries, and anomalies, preprocessing ensures that the dataset is clean, structured, and optimized for accurate machine learning predictions. Without proper preprocessing, the fraud detection model may generate biased results or fail to identify fraudulent activities effectively. This phase involves handling null values, removing duplicate transactions, addressing outliers, and encoding categorical data to make it suitable for machine learning algorithms.

Handling Null Values Using Removal and Imputation Techniques

In real-world UPI transaction datasets, missing values can arise due to technical issues, incomplete data entries, or system failures during transaction processing. These missing values, if not handled correctly, can introduce bias and reduce model performance. If a column has a large percentage of missing data, it may be dropped entirely. However, if missing values are minimal, imputation techniques are used to fill them. **For numerical fields like transaction amount, median imputation is applied, while categorical columns such as transaction type are filled using mode imputation.** Additionally, forward-fill techniques are used for time-series transactional data, ensuring that the sequence remains intact.

Removing Duplicate Entries to Prevent Biased Model Predictions

Duplicate transactions occur when the same transaction is recorded multiple times due to API failures, system errors, or network delays. If these duplicate entries are not removed, they can distort the model by overrepresenting certain patterns, leading to biased predictions. To detect duplicates, key attributes such as transaction ID, timestamp, sender-receiver details, and transaction amount are compared. **Exact duplicates are dropped immediately, while partial duplicates (where transaction details match except for minor variations in timestamps) are carefully analyzed before removal.** This ensures that legitimate transactions are not mistakenly eliminated while preventing data redundancy.

Handling Outliers to Improve Model Accuracy

Outliers in UPI transaction data often indicate potential fraudulent activities, such as unusually large transactions, frequent microtransactions, or transactions occurring at odd hours. If left unaddressed, these extreme values can mislead the model, making it difficult to distinguish between normal and fraudulent transactions. **Statistical methods such as boxplot analysis, Z-score analysis, and clustering techniques (e.g., DBSCAN) are used to detect and handle outliers.** Boxplots help identify extreme values, while Z-score filtering removes transactions that significantly deviate from the mean. Instead of blindly removing all outliers, careful assessment is done to retain meaningful fraud indicators.

Encoding Categorical Variables for Machine Learning Compatibility

UPI transaction datasets contain categorical data such as transaction type (e.g., "bill payment," "fund transfer"), merchant category, and fraud labels ("genuine" or "fraud"). Since machine learning models work best with numerical data, categorical variables must be converted into numerical formats. **Label**

encoding is used for binary categories, while one-hot encoding is applied to multi-category fields. This transformation ensures that categorical data is properly represented in the model without introducing biases. After encoding, the dataset is structured and ready for the next phase of fraud detection, where machine learning models are trained to identify fraudulent patterns.

6.3 Exploratory Data Analysis (EDA)

- Displot – Used to analyze the distribution of numerical features.
- Scatter Plot – Helps visualize relationships between different transaction attributes.
- Boxplot – Detects outliers in transaction amounts and frequency.
- Heatmap – Identifies feature correlations using a color-coded matrix.

Exploratory Data Analysis (EDA) plays a vital role in understanding transaction data, identifying patterns, and detecting anomalies that may indicate fraudulent activities. By using various visualization techniques, we can gain insights into the distribution of transaction amounts, relationships between attributes, and the presence of outliers. EDA helps refine the dataset and improve model performance by identifying important features that contribute to fraud detection.

Displot – Analyzing the Distribution of Numerical Features

A displot (distribution plot) helps visualize how numerical features, such as transaction amounts and frequency, are distributed across the dataset. This is crucial in fraud detection, as fraudulent transactions often exhibit unusual patterns compared to normal transactions. For instance, genuine transactions typically follow a smooth distribution with most transactions clustered within a reasonable range. In contrast, fraudulent transactions may show sharp spikes in high-value transactions or an unusual frequency of small transactions. By plotting the distribution of transaction amounts, we can observe whether the dataset is skewed

and whether fraud cases exhibit unique distribution patterns. If the distribution is highly skewed, transformations such as logarithmic scaling may be applied to normalize the data for better model performance.

Scatter Plot – Visualizing Relationships Between Transaction Attributes

A scatter plot is useful for analyzing relationships between different transaction attributes, such as transaction amount versus transaction time or sender-receiver transaction frequency. In fraud detection, scatter plots help identify suspicious clusters or trends that deviate from normal behavior. For instance, a scatter plot of transaction amount versus time might reveal that fraudulent transactions frequently occur at odd hours when fewer security checks are in place. Similarly, plotting sender ID against transaction frequency can help detect users who send or receive an abnormally high number of transactions within a short period—often a sign of fraudulent activity. By examining scatter plots, financial institutions can detect unusual transaction behaviors that warrant further investigation.

Boxplot – Detecting Outliers in Transaction Amounts and Frequency

A boxplot is an effective tool for detecting outliers in transaction data, which are often strong indicators of fraud. In a typical transaction dataset, most transactions fall within a normal range, but fraudulent transactions tend to have extreme values. A boxplot helps visualize the spread of transaction amounts and identify transactions that exceed the expected limits. For instance, in a UPI fraud dataset, fraudulent transactions may appear as extreme points above the upper whisker of the boxplot, representing significantly high transaction amounts. Additionally, boxplots can be used to analyze transaction frequency per user, helping to flag accounts with unusually high activity. By identifying and handling these outliers, we improve the model's ability to detect fraud without being misled by extreme variations.

Heatmap – Identifying Feature Correlations Using a Color-Coded Matrix

A heatmap is a graphical representation of feature correlations, where colors indicate the strength of relationships between different variables. In fraud detection, understanding these correlations helps determine which features contribute the most to fraudulent transactions. For example, if transaction amount is highly correlated with fraud occurrences, it suggests that large transactions have a higher chance of being fraudulent. Similarly, correlations between transaction frequency and fraud cases can reveal whether fraudsters perform multiple small transactions to evade detection. By analyzing the heatmap, we can select the most relevant features for the machine learning model, reducing noise and improving prediction accuracy. A strong correlation between specific attributes can also indicate patterns that fraudsters frequently exploit, enabling proactive security measures in the UPI system.

6.4 Splitting of Data

- Training Set (80%) – Used to train the machine learning models.
- Testing Set (20%) – Used to evaluate model performance on unseen data.

Splitting the dataset into **training and testing sets** is a crucial step in building an effective fraud detection model. It ensures that the model learns patterns from one portion of the data and is evaluated on unseen data to test its generalization capability. A well-balanced split prevents overfitting, where the model memorizes the training data instead of learning meaningful fraud detection patterns. In this project, we divide the dataset into **80% for training and 20% for testing.** This split helps in optimizing model performance while maintaining a sufficient amount of data for evaluation.

Training Set (80%) – Model Learning and Fraud Pattern Identification

The training set consists of 80% of the dataset and is used to train the machine learning models, including Random Forest, AdaBoost, CatBoost,

XGBoost, and LightGBM. During training, these models learn to recognize fraudulent patterns by analyzing various transaction attributes such as transaction amount, frequency, sender-receiver details, and timestamps. The model identifies distinguishing characteristics between normal and fraudulent transactions, adjusting its internal parameters to maximize accuracy. Feature engineering techniques, such as scaling numerical features and encoding categorical variables, are applied to optimize the learning process. The training process also includes handling class imbalances, as fraudulent transactions are typically much fewer than genuine ones, requiring techniques like oversampling or cost-sensitive learning.

Testing Set (20%) – Evaluating Model Performance on Unseen Transactions

The testing set comprises the remaining 20% of the data and is used to evaluate the trained model's performance. Since the model has never seen this data before, the test set serves as an independent validation, ensuring that the model can accurately detect fraud in real-world scenarios. The evaluation includes measuring key performance metrics such as accuracy, precision, recall, F1-score, and the confusion matrix. These metrics help determine whether the model can effectively differentiate between legitimate and fraudulent transactions without excessive false positives or negatives. By analyzing the testing results, we can fine-tune hyperparameters and select the best-performing model for deployment in the UPI fraud detection system.

6.5 Model Deployment

- Random Forest – A robust ensemble learning method for classification.
- AdaBoost – Boosting algorithm that improves weak classifiers.
- CatBoost – Optimized for categorical data and faster training.
- XGBoost – Highly efficient gradient boosting model.
- LightGBM – A fast and scalable boosting algorithm for large datasets.

Deploying a fraud detection model in a UPI system is the final step in building a **real-time fraud prevention mechanism.** This phase involves selecting the best-performing model and integrating it with the web-based application. To ensure optimal fraud detection, multiple machine learning models are evaluated, each offering unique advantages. The deployment process includes **saving the trained model, integrating it into the web framework, and enabling real-time transaction analysis.** The models used in this project include **Random Forest, AdaBoost, CatBoost, XGBoost, and LightGBM**, each contributing to enhanced fraud detection through different methodologies.

Random Forest – A Robust Ensemble Learning Method for Classification

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their outputs for more stable and accurate fraud detection. It is highly effective in this project because it can handle large transaction datasets and detect complex fraud patterns. The algorithm works by randomly selecting subsets of transaction attributes and building multiple decision trees, reducing the risk of overfitting. Random Forest is particularly useful for fraud detection as it provides high accuracy and interpretability, allowing financial institutions to understand why a transaction was flagged as fraudulent.

AdaBoost – Boosting Algorithm That Improves Weak Classifiers

AdaBoost (Adaptive Boosting) is a boosting algorithm that enhances weak classifiers by giving more importance to misclassified transactions in each iteration. In fraud detection, it helps improve detection rates by focusing on fraudulent cases that other models might overlook. The model assigns higher weights to transactions that were previously misclassified, forcing subsequent iterations to prioritize them. This approach ensures that the fraud detection system does not rely solely on the most common fraud patterns but also identifies subtle

fraud attempts. AdaBoost is particularly useful when dealing with imbalanced datasets, where fraudulent transactions are significantly fewer than legitimate ones.

CatBoost – Optimized for Categorical Data and Faster Training

CatBoost (Categorical Boosting) is specifically designed to handle categorical variables efficiently, making it ideal for fraud detection systems where transaction details involve non-numeric data such as sender-receiver IDs and transaction types. Unlike traditional boosting algorithms, CatBoost automatically encodes categorical features without requiring extensive preprocessing, reducing training time and improving performance. Additionally, CatBoost prevents overfitting, ensuring that the model remains generalizable to new fraudulent activities. Its ability to work effectively with large datasets and categorical features makes it a strong candidate for fraud detection in UPI transactions.

XGBoost – Highly Efficient Gradient Boosting Model

XGBoost (Extreme Gradient Boosting) is a powerful gradient boosting model known for its speed and efficiency in large-scale fraud detection tasks. It applies regularization techniques to prevent overfitting, ensuring that the model generalizes well to unseen transactions. XGBoost excels in detecting fraud by identifying subtle patterns in transactional data, making it highly effective in differentiating between legitimate and fraudulent transactions. It is widely used in real-world fraud detection systems due to its ability to handle missing data, work with large datasets, and provide high accuracy.

LightGBM – A Fast and Scalable Boosting Algorithm for Large Datasets

LightGBM (Light Gradient Boosting Machine) is designed for scalability and efficiency, making it an excellent choice for real-time fraud detection in high-volume UPI transactions. Unlike traditional boosting methods, LightGBM splits

trees leaf-wise instead of depth-wise, improving performance and reducing memory usage. This makes it well-suited for handling large-scale transaction datasets without compromising speed. In a UPI fraud detection system, LightGBM ensures rapid fraud analysis, allowing the system to flag suspicious transactions in real time.

Integration and Deployment

Once the best-performing model is selected based on accuracy and evaluation metrics, it is saved and integrated into the web framework. The model is then used to analyze live transactions, predicting fraudulent behavior based on learned patterns. The deployment involves creating an API that allows the UPI system to send transaction data to the model, process it, and return a fraud prediction. Users interact with the web interface to monitor flagged transactions, enhancing security in the digital payment ecosystem.

6.6 Model Evaluation, Model Deployment & User Interaction (GUI)

- Accuracy Score – Measures the proportion of correctly classified transactions.
- Confusion Matrix – Evaluates the number of true positives, false positives, true negatives, and false negatives.
- Precision, Recall, and F1-Score – Helps determine model efficiency in fraud detection.
- The best-performing model is selected based on evaluation metrics.
- The model is saved and integrated into a Flask-based web framework.
- Run the app.py file to start the web application.
- Copy the generated local server link and open it in Chrome.
- Enter transaction details in the UI and click "Predict".

- The system analyzes the input and displays the fraud prediction on the screen.

Ensuring the effectiveness of a fraud detection system requires rigorous evaluation, careful deployment, and an intuitive user interface for real-time fraud analysis. The evaluation process focuses on assessing the model's accuracy, precision, recall, F1-score, and confusion matrix to determine its reliability in detecting fraudulent transactions. Once the most effective model is identified, it is deployed in a Flask-based web framework, allowing users to interact with the system through a simple UI. The integration of machine learning with a user-friendly interface ensures that fraud detection is both efficient and accessible for financial institutions and end users.

Accuracy Score – Evaluating Model Performance

The accuracy score is a key metric in measuring how well the model classifies transactions as either fraudulent or legitimate. It represents the proportion of correctly predicted transactions out of the total transactions processed. While accuracy is a useful indicator, it may not be the best standalone metric, especially in imbalanced datasets where fraudulent transactions are significantly fewer than legitimate ones. For this reason, additional evaluation metrics are considered to ensure that the model is not biased toward the majority class.

Confusion Matrix – Understanding Model Predictions

The confusion matrix provides deeper insights into model performance by categorizing predictions into four groups:

- **True Positives (TP) –** Fraudulent transactions correctly identified as fraud.
- **False Positives (FP) –** Legitimate transactions incorrectly flagged as fraud.
- **True Negatives (TN) –** Legitimate transactions correctly identified as non-fraudulent.

- **False Negatives (FN) –** Fraudulent transactions mistakenly classified as legitimate.

A well-performing model minimizes false positives and false negatives, ensuring that fraudulent transactions do not go undetected while also reducing unnecessary disruptions for genuine users. By analyzing the confusion matrix, financial institutions can fine-tune the model to improve its real-world fraud detection capability.

Precision, Recall, and F1-Score – Measuring Detection Efficiency

Since fraud detection requires minimizing false negatives (missed fraud cases) while avoiding excessive false positives (unnecessary transaction blocks), three important metrics are used:

- **Precision –** Measures the proportion of correctly flagged fraudulent transactions out of all flagged cases. A high precision means fewer false alarms.
- **Recall –** Indicates how well the model detects actual fraud cases. A high recall ensures that most fraudulent transactions are identified.
- **F1-Score –** A balance between precision and recall, providing an overall assessment of fraud detection efficiency.

These metrics help in selecting the best-performing model, ensuring that it effectively minimizes financial losses while maintaining a seamless user experience.

Model Deployment – Flask Integration for Real-Time Predictions

Once the model is finalized, it is saved and integrated into a Flask-based web application. Flask provides a lightweight and efficient framework for handling transaction data, processing fraud detection requests, and displaying real-time predictions. The model deployment process involves:

- Loading the trained model and setting up an API for transaction analysis.
- Running the **app.py** file to launch the web application.
- Generating a local server link for user interaction.

User Interaction – Predicting Fraudulent Transactions in the Web Interface

Users interact with the fraud detection system through a simple and intuitive web interface. The process involves:

- Opening the generated local server link in Chrome.
- Entering transaction details in the UI, including amount, sender-receiver details, and time of transaction.
- Clicking the "Predict" button, which sends the transaction data to the model.
- Receiving real-time fraud analysis, where the system displays whether the transaction is legitimate or fraudulent.

CHAPTER : 7

RESULTS AND DISCUSSION

7. Results and Discussion

- Model Performance Summary
- Comparison of Algorithm Accuracy
- Implications of Results
- Limitations and Challenges

7.1 Model Performance Summary

Evaluating the **performance of machine learning models** in fraud detection is crucial to ensuring the accuracy and reliability of the system. The performance of each algorithm is assessed based on key metrics such as **accuracy, precision, recall, F1-score, and confusion matrix**. These metrics help determine how well the model classifies transactions as either fraudulent or legitimate. A well-performing model should have **high precision to minimize false positives and high recall to reduce false negatives**.

Each algorithm undergoes extensive **training and testing** on the dataset, with hyperparameter tuning applied to improve results. The dataset is split into **80% training and 20% testing**, ensuring that the model generalizes well to new, unseen data. Additionally, **cross-validation** techniques are employed to validate model robustness and prevent overfitting. By analyzing these results, we can select the best algorithm that balances **efficiency, accuracy, and adaptability** in real-world fraud detection scenarios.

Beyond numerical accuracy, the **scalability and speed** of the model are also considered. Fraud detection systems must **process transactions in real-time**, making computational efficiency an essential factor. A model that performs well but takes too long to analyze transactions may not be suitable for deployment in high-frequency financial systems like **UPI (Unified Payments Interface)**. Therefore, model selection is not just about accuracy but also **speed and adaptability**.

Ultimately, the **goal of performance evaluation** is to ensure that the fraud detection system can effectively minimize **financial losses and security risks**. By comparing different models and assessing their strengths and weaknesses, we develop a **robust, scalable, and highly accurate fraud detection system** that provides financial institutions with a reliable security mechanism.

7.2 Comparison of Algorithm Accuracy

Various machine learning algorithms are tested to determine their **accuracy and efficiency** in detecting fraudulent transactions. The models include **Random Forest, AdaBoost, CatBoost, XGBoost, and LightGBM**, each offering unique advantages in fraud detection. Random Forest, being an ensemble model, performs well with **high accuracy and low variance**, while boosting algorithms like XGBoost and LightGBM provide superior results in handling **imbalanced datasets**.

When comparing accuracy, **XGBoost and LightGBM** often outperform other models due to their ability to handle large datasets and **optimize computational speed**. These models utilize gradient boosting techniques to **improve performance iteratively**, learning from misclassified transactions in previous iterations. **AdaBoost and CatBoost** also deliver competitive accuracy but may require additional tuning to handle the **complex nature of fraudulent transactions**.

Although accuracy is an important metric, it is not the only deciding factor. In fraud detection, **precision and recall are equally important**. For example, a model with high accuracy but **low recall** may fail to detect a large number of fraudulent transactions. On the other hand, a model with **high recall but low precision** may flag too many legitimate transactions as fraud, causing inconvenience to users. Therefore, a balance between **accuracy, precision, and recall** must be maintained to ensure an effective fraud detection system.

Through **comparative analysis**, it is evident that **ensemble learning models** such as XGBoost and LightGBM offer the best trade-off between **accuracy, efficiency, and computational performance**. These models are thus preferred for real-time fraud detection in the UPI framework, ensuring a **secure and seamless transaction experience** for users.

7.3 Implications of Results

The results from the model evaluation have significant **implications for fraud detection systems** in digital payments. By identifying the most effective algorithms, financial institutions can **enhance transaction security** and **reduce financial losses due to fraud**. An accurate fraud detection system not only improves **trust in digital transactions** but also encourages more users to adopt **UPI-based payments** with confidence.

A key implication is the ability to **detect fraud patterns in real-time**. Since fraudulent transactions often exhibit specific behavioral patterns, a well-trained machine learning model can **analyze transaction attributes instantly** and flag suspicious activity. This helps financial institutions take **immediate action**, either by **blocking transactions, notifying users, or requesting additional verification**. The ability to **adapt to new fraud patterns** is a major advantage of using machine learning over traditional rule-based systems.

Another important implication is the **balance between security and user convenience**. While fraud detection models must be strict in identifying suspicious transactions, **excessively aggressive models can disrupt legitimate transactions** by falsely flagging them as fraudulent. This could lead to **frustration among users and increased operational costs** for financial institutions due to additional manual verification processes. Therefore, fraud detection systems must **strike a balance** between security and efficiency,

ensuring **seamless transactions for genuine users** while blocking fraudulent ones.

Finally, the **insights gained from fraud detection models** can be used to **refine security policies and regulations** for digital transactions. By continuously analyzing fraud trends, financial institutions and regulatory bodies can **implement proactive security measures**, such as **enhanced authentication methods, dynamic risk assessment, and better user education on fraud awareness**. These measures help create a **stronger and more resilient financial ecosystem**.

7.4 Limitations and Challenges

Limitations

One of the primary **limitations of fraud detection models** is the **imbalance in the dataset**. In financial transactions, fraudulent cases make up a very **small fraction** of the total transactions, often leading to a bias in model predictions. Since most machine learning algorithms optimize for overall accuracy, they may misclassify fraudulent transactions as legitimate due to the overwhelming presence of non-fraudulent transactions. To mitigate this, techniques like **Synthetic Minority Over-sampling Technique (SMOTE)** or **cost-sensitive learning** are used, but these methods do not always fully resolve the issue, making fraud detection an ongoing challenge.

Another significant limitation is the **difficulty in detecting evolving fraud patterns**. Fraudsters continuously **modify their tactics** to bypass detection mechanisms, making it hard for machine learning models to keep up. Traditional models rely on historical data, and if fraudsters introduce new strategies that are not present in the training data, the model may fail to recognize them. This necessitates **continuous model retraining**, which can be resource-intensive and may still lag behind real-time fraud advancements.

The **computational complexity** of fraud detection models presents another limitation. Models like **XGBoost, LightGBM, and deep learning architectures** require substantial computing power to analyze large-scale financial data. Running these models in real-time for **millions of transactions per second** in UPI systems is challenging. While cloud-based deployment and distributed computing can help, maintaining efficiency without compromising **speed and accuracy** remains a key concern.

False positives are another drawback of fraud detection models. While high recall models aim to capture **all fraudulent transactions**, they may also flag many **legitimate transactions** as fraud. This can lead to **user dissatisfaction** and unnecessary manual verification efforts by financial institutions. If users frequently experience transaction blocks due to false positives, they may lose trust in the **UPI payment system**, leading to financial institutions balancing **fraud prevention with customer convenience**.

Finally, **data privacy and security concerns** pose a significant limitation. Fraud detection models analyze sensitive transaction data, which must be **secured and anonymized** to prevent misuse. Strict **data protection laws** such as **GDPR (General Data Protection Regulation) and India's Personal Data Protection Bill** mandate that personal financial data must be handled responsibly. Failure to comply with these regulations can lead to **legal penalties and reputational damage** for financial institutions.

Challenges

One of the biggest challenges in fraud detection is **real-time transaction analysis**. In the UPI system, transactions are processed **instantly**, leaving little time for fraud detection models to analyze and flag suspicious activity. Unlike credit card transactions that may take minutes to verify, UPI payments require

immediate fraud detection, making it crucial to develop **fast and efficient machine learning models** that do not delay the user experience.

Another challenge is **handling large-scale data efficiently**. Fraud detection systems must process **millions of transactions daily**, each containing multiple attributes like transaction amount, location, device ID, and time. Storing, processing, and analyzing this high-volume data in real-time requires **advanced infrastructure**, including **high-performance computing (HPC) environments, cloud storage, and distributed databases**. Many organizations struggle with implementing such infrastructure cost-effectively.

The **dynamic nature of fraud** is another pressing challenge. Fraudsters use sophisticated methods such as **AI-generated fake accounts, coordinated fraud rings, and device spoofing** to bypass detection. Machine learning models trained on past fraud patterns may not be able to detect **new fraud schemes**, requiring financial institutions to **frequently update** their fraud detection algorithms. This ongoing process requires dedicated **data scientists, AI engineers, and fraud analysts**, increasing operational costs.

A major challenge is the **interpretability of machine learning models**. While complex models like **deep learning and ensemble algorithms** provide higher accuracy, they often function as **black boxes**, making it difficult for fraud analysts to understand why a transaction was flagged as fraudulent. Regulatory bodies require **explainability in AI-driven fraud detection**, and institutions must adopt techniques like **SHAP (SHapley Additive Explanations) and LIME (Local Interpretable Model-agnostic Explanations)** to ensure transparency in decision-making.

Finally, **user resistance and regulatory challenges** create obstacles in fraud detection implementation. Some users **hesitate to share transaction details** due to privacy concerns, reducing the amount of data available for training

fraud detection models. Additionally, regulatory compliance laws vary by country, making it difficult for **global financial institutions** to create a standardized fraud detection system. Institutions must **align their fraud prevention strategies** with government policies while ensuring they do not violate customer privacy rights.

Addressing these challenges requires **continuous innovation, collaboration between financial institutions and regulatory bodies, and advancements in AI-driven fraud detection techniques**. By overcoming these barriers, financial organizations can **enhance fraud prevention measures while maintaining user trust in digital payment systems**.

CHAPTER : 8

CONCLUSION

8. Conclusion

- Key Findings
- Recommendations for Future Work

8.1 Key Findings

One of the most significant findings from our project is that **machine learning models can effectively detect fraudulent UPI transactions** in real-time. Through rigorous experimentation with various algorithms, we observed that ensemble methods such as **Random Forest, XGBoost, and LightGBM** delivered superior accuracy in identifying fraudulent activities compared to traditional rule-based detection. The ability of these models to capture **non-linear relationships and subtle transaction patterns** has significantly enhanced fraud detection precision, reducing false positives and false negatives. This demonstrates that data-driven fraud detection systems are far more efficient than conventional manual verification processes.

Another crucial finding is the **importance of data preprocessing in improving model performance**. Our analysis revealed that handling **missing values, removing duplicate records, encoding categorical variables, and scaling numerical features** played a pivotal role in enhancing the accuracy of fraud detection. Additionally, detecting and addressing outliers in transaction amounts significantly improved model reliability. This underscores the fact that **high-quality data preparation is just as critical as model selection** when developing fraud detection systems.

Our study also highlighted the **dynamic nature of fraudulent transactions**, which evolve rapidly with new fraud techniques emerging constantly. Static rule-based fraud detection systems are unable to keep pace with these changes, whereas machine learning models—when periodically updated with **new fraudulent patterns and retrained using fresh data**—can adapt and

improve detection accuracy. This finding stresses the necessity of **continuous learning and model retraining** to ensure fraud detection systems remain effective against emerging threats.

Finally, we found that **deploying fraud detection models into a real-world environment presents both technical and operational challenges**. While our model performed exceptionally well during testing, real-world scenarios introduce complexities such as **scalability, latency issues, and integration with existing banking infrastructure**. Additionally, false positive alerts, though minimized, still need to be addressed efficiently to **balance fraud prevention with user experience**. These findings emphasize the need for **ongoing refinement and optimization** of fraud detection models post-deployment.

8.2 Recommendations for Future Work

To further enhance the effectiveness of UPI fraud detection, we recommend **expanding the dataset** to include real-time transaction records from multiple financial institutions. A larger and more diverse dataset will help the model learn **broader fraud patterns**, improving its ability to detect sophisticated fraud attempts. Additionally, leveraging **synthetic data generation techniques** could help mitigate class imbalance issues by creating artificial fraudulent transactions to enrich the dataset.

Another area for improvement is **the integration of deep learning models**, such as **Long Short-Term Memory (LSTM) networks and Transformer-based architectures**, which have proven effective in sequence-based fraud detection tasks. Unlike traditional models, deep learning techniques can analyze complex sequential transaction patterns and detect anomalies more efficiently. Implementing these advanced techniques could lead to **further reductions in false positives and improved fraud detection accuracy**.

We also recommend implementing **real-time fraud detection with automated alert systems**. By integrating fraud detection models with **real-time monitoring tools and automated intervention mechanisms**, financial institutions can **immediately flag suspicious transactions** and take preventive action before fraud occurs. Additionally, introducing an **explainability framework** such as **SHAP (SHapley Additive Explanations)** will help make fraud detection decisions more transparent, allowing banking institutions to understand why a transaction was flagged as fraudulent.

Finally, future research should focus on **enhancing the security and privacy aspects** of fraud detection models. Since fraud detection relies on sensitive transaction data, implementing **privacy-preserving machine learning techniques** such as **federated learning and homomorphic encryption** can help maintain **data confidentiality while improving fraud detection capabilities**. This would ensure compliance with **regulatory standards** while maintaining high detection accuracy in financial transactions.

CHAPTER : 9

REFERENCES

9. References

[1] Gupta, R., Srivastava, P., Taluja, H. K., Sharma, S., Samant, S., Ratna, S., & Sharma, A. (2023, March). Leveraging Machine Learning Algorithms for Fraud Detection and Prevention in Digital Payments: A Cross Country Comparison. In International Conference on Information Technology (pp. 369-381). Singapore: Springer Nature Singapore.

[2] Bhuvaneswari, R., Vasanthi, V., Jerald, M. P. A., & Franklin, I. B. (2023). ANALYSIS OF ONLINE INTRUSION DETECTION MODELS TO INCORPORATE SECURED DIGITAL CASH TRANSACTION IN MOBILE

SMART SYSTEMS. ICTACT Journal on Communication Technology, 14(4).

[3] Kadam, M. K. D., Omanna, M. M. R., Neje, M. S. S., & Nandai, M. S. S. Online Transactions Fraud Detection using Machine Learning.

[4] Tiwari, P., & Garg, P. (2023). Fraud Risk Management System.

[5] Singla, J. (2020, June). A survey of deep learning based online transactions fraud detection systems. In 2020 International Conference on Intelligent Engineering and Management (ICIEM) (pp. 130-136). IEEE.

[6] Sangeetha, R., Joy, R. P., Denisha, M., & Dhas, J. P. M. (2023). An Innovation Detection of Vulnerabilities for Digital Transactions in Financial Institutions Using Cyber Security Framework. International Journal of Intelligent Systems and Applications in Engineering, 11(3), 70-76.

[7] Kumar, K. M. (2023). DIGITAL BANKING EMPOWERMENT: UNVEILING NOVEL STRATEGIES TO SAFEGUARD ONLINE TRANSACTIONS FROM FRAUDULENT ACTIVITIES. The Online Journal of Distance Education and e-Learning, 11(2).

[8] Akinje, A. O., & Fuad, A. (2021). Fraudulent Detection Model Using Machine Learning Techniques for Unstructured Supplementary Service Data. International Journal of Innovative Computing, 11(2), 51-60.

[9] Jain, S. (2023). Understanding Social Engineering and it's impact on Merchant based UPI frauds.

[10] Priya, N., Ahmed, J., & Alam, M. A. Digital Payments: A scheme for Fraud Data Collection and Use in Indian Banking Sector.

APPENDIX

10. Appendix

CODE SNIPPETS

************IMPORTING LIBRARIES************

```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
import plotly.graph_objs as go
import plotly.figure_factory as ff
from plotly import tools
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
init_notebook_mode(connected=True)
import gc
from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from catboost import CatBoostClassifier
```

```
from sklearn import svm
import lightgbm as lgb
from lightgbm import LGBMClassifier
import xgboost as xgb
pd.set_option('display.max_columns', 100)
RFC_METRIC = 'gini'  #metric used for RandomForrestClassifier
NUM_ESTIMATORS = 100 #number of estimators used for RandomForrestClassifier
NO_JOBS = 4 #number of parallel jobs used for RandomForrestClassifier
#TRAIN/VALIDATION/TEST SPLIT
#VALIDATION
VALID_SIZE = 0.20 # simple validation using train_test_split
TEST_SIZE = 0.20 # test size using_train_test_split
#CROSS-VALIDATION
NUMBER_KFOLDS = 5 #number of KFolds for cross-validation
RANDOM_STATE = 2018
MAX_ROUNDS = 1000 #lgb iterations
EARLY_STOP = 50 #lgb early stop
OPT_ROUNDS = 1000  #To be adjusted based on best validation rounds
VERBOSE_EVAL = 50 #Print out metric result
IS_LOCAL = False
import os
```

```
PATH="E:\Saaswath\credit card fraud detection"
```

************READ THE DATASET************

```
data_df = pd.read_csv(PATH+"/creditcard.csv")
```

************COLAB************

```
total = data_df.isnull().sum().sort_values(ascending = False)
percent = (data_df.isnull().sum()/data_df.isnull().count()*100).sort_values(ascending = False)
pd.concat([total, percent], axis=1, keys=['Total', 'Percent']).transpose()
temp = data_df["Class"].value_counts()
df = pd.DataFrame({'Class': temp.index,'values': temp.values})

trace = go.Bar(
    x = df['Class'],y = df['values'],
    name="Credit Card Fraud Class - data unbalance (Not fraud = 0, Fraud = 1)",
    marker=dict(color="Red"),
    text=df['values']
)
data = [trace]
layout = dict(title = 'Credit Card Fraud Class - data unbalance (Not fraud = 0, Fraud = 1)',
```

```
        xaxis = dict(title = 'Class', showticklabels=True),
        yaxis = dict(title = 'Number of transactions'),
        hovermode = 'closest',width=600
       )
fig = dict(data=data, layout=layout)
iplot(fig, filename='class')
tmp = pd.DataFrame({'Feature': predictors, 'Feature importance': clf.feature_importances_})
tmp = tmp.sort_values(by='Feature importance',ascending=False)
plt.figure(figsize = (7,4))
plt.title('Features importance',fontsize=14)
s = sns.barplot(x='Feature',y='Feature importance',data=tmp)
s.set_xticklabels(s.get_xticklabels(),rotation=90)
plt.show()
cm = pd.crosstab(valid_df[target].values, preds, rownames=['Actual'], colnames=['Predicted'])
fig, (ax1) = plt.subplots(ncols=1, figsize=(5,5))
sns.heatmap(cm,
          xticklabels=['Not Fraud', 'Fraud'],
          yticklabels=['Not Fraud', 'Fraud'],
          annot=True,ax=ax1,
          linewidths=.2,linecolor="Darkblue", cmap="Blues")
```

```
plt.title('Confusion Matrix', fontsize=14)
plt.show()
clf = AdaBoostClassifier(random_state=RANDOM_STATE,
                         algorithm='SAMME.R',
                         learning_rate=0.8,
                            n_estimators=NUM_ESTIMATORS)
# Prepare the train and valid datasets
dtrain = xgb.DMatrix(train_df[predictors], train_df[target].values)
dvalid = xgb.DMatrix(valid_df[predictors], valid_df[target].values)
dtest = xgb.DMatrix(test_df[predictors], test_df[target].values)

#What to monitor (in this case, **train** and **valid**)
watchlist = [(dtrain, 'train'), (dvalid, 'valid')]

# Set xgboost parameters
params = {}
params['objective'] = 'binary:logistic'
params['eta'] = 0.039
params['silent'] = True
params['max_depth'] = 2
params['subsample'] = 0.8
params['colsample_bytree'] = 0.9
```

```
params['eval_metric'] = 'auc'
params['random_state'] = RANDOM_STATE
```

************RUN MODEL************

```
evals_results = {}
model = lgb.train(params,
                  dtrain,
                  valid_sets=[dtrain, dvalid],
                  valid_names=['train','valid'],
                  evals_result=evals_results,
                  num_boost_round=MAX_ROUNDS,
                  early_stopping_rounds=2*EARLY_STOP,
                  verbose_eval=VERBOSE_EVAL,
                  feval=None)
```

************PYTHON FILE************

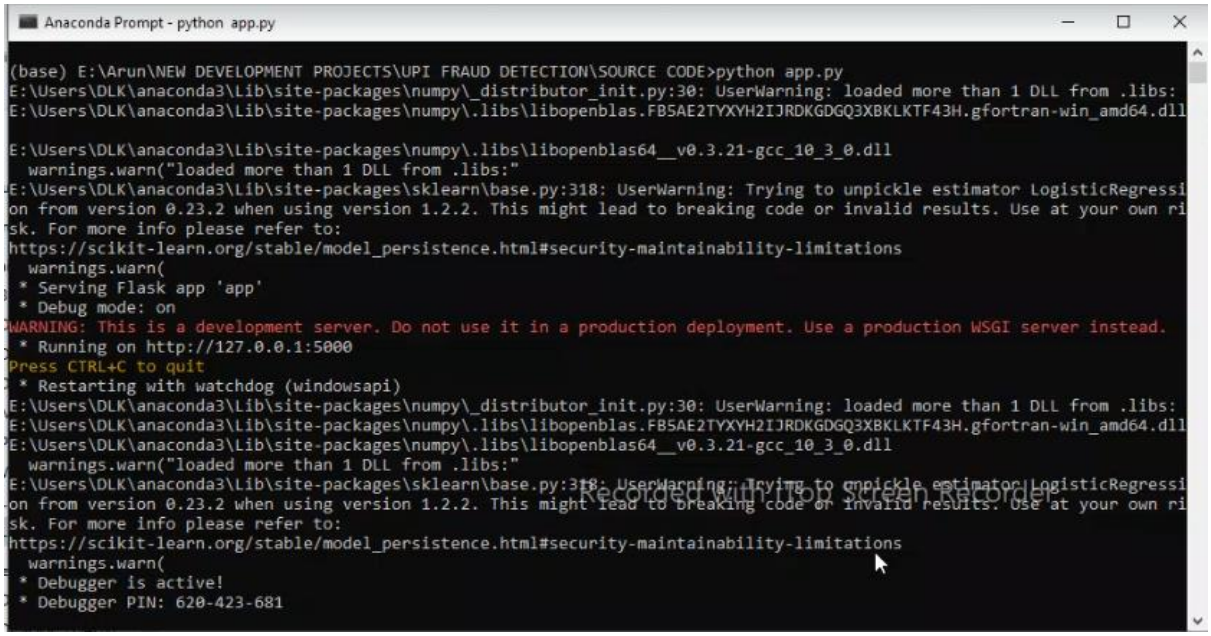
************app.py************

```
from flask import Flask, render_template, url_for, request
import pandas as pd, numpy as np
import pickle
# load the model from disk
filename = 'model.pkl'
clf = pickle.load(open(filename, 'rb'))
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return render_template('home.html')
@app.route('/predict', methods = ['POST'])
def predict():
    if request.method == 'POST':
        me = request.form['message']
        message = [float(x) for x in me.split()]
        vect = np.array(message).reshape(1, -1)
        my_prediction = clf.predict(vect)
    return render_template('result.html',prediction = my_prediction)
if __name__ == '__main__':
    app.run(debug=True)
```

ScreenShots

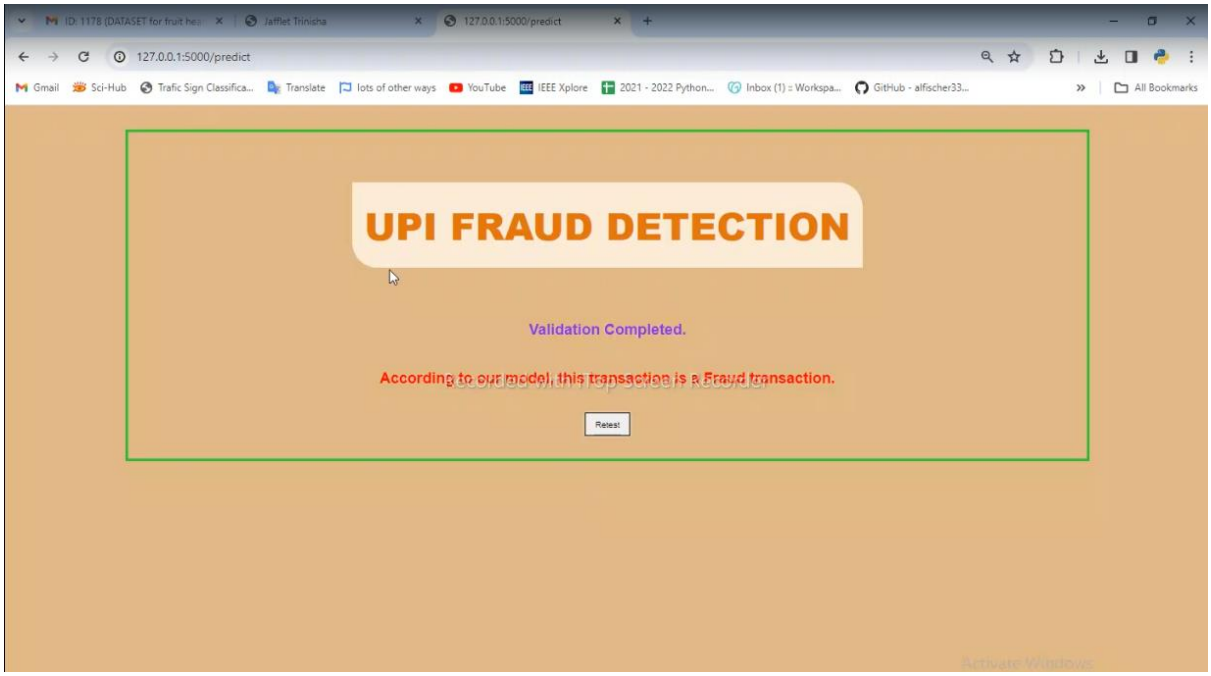
SS.1 : Build Successful



SS.2 : Home Page

SS.3 : Test1

SS.4 : Result1



SS.5 : Test2

SS.6 : Result2